

Java Multithreading

Complete Interview Preparation Guide

60+ Interview Questions • Cheat Sheet • Best Practices • Code Examples

Covers: Threads • Synchronization • Locks • Executors • CompletableFuture • Java Memory Model

Table of Contents

Table of Contents.....	1
Section 1: Multithreading Fundamentals.....	2
1. Extending Thread class.....	2
2. Implementing Runnable interface (Preferred).....	3
3. Lambda (Java 8+).....	3
4. Callable + Future (returns a value).....	3
Section 2: Synchronization.....	5
Synchronized instance method – lock on 'this'.....	5
Synchronized static method – lock on Class object.....	5
Synchronized block – finer-grained, preferred.....	5
Section 3: java.util.concurrent Utilities.....	7
CountDownLatch – wait for N events to complete.....	9
CyclicBarrier – all threads wait at a common point.....	9
Semaphore – control concurrent access count.....	9
Section 4: Executor Framework & Thread Pools.....	9
Section 5: Future, CompletableFuture & Async Programming.....	11
Creating.....	11
Transforming.....	11
Combining.....	12
Error Handling.....	12
Section 6: Classic Multithreading Problems.....	12
Using BlockingQueue (Recommended).....	12
Double-Checked Locking (Java 5+, requires volatile).....	13
Initialization-on-demand holder (Best – lazy + thread-safe + no sync).....	13

Section 7: Advanced Topics.....	14
Section 8: Common Mistakes & Best Practices.....	18
8.1 Common Mistakes.....	18
8.2 Best Practices Summary.....	18
Section 9: Quick-Fire Interview Questions (Q41–Q60).....	18
Cheat Sheet: Java Multithreading at a Glance.....	21
Thread Creation Methods.....	21
Synchronization Toolkit.....	21
Thread Pool Quick Reference.....	22
Key Terms & Formulas.....	22
Pattern Cheat Sheet.....	22
Interview Tips.....	23

Section 1: Multithreading Fundamentals

★ **FREQUENTLY ASKED** Q1. What is a thread in Java? How is it different from a process?

Answer:

A **thread** is the smallest unit of execution within a process. Multiple threads share the same process memory space (heap, static variables, open files) while each has its own **stack, program counter, and local variables**.

Key differences:

Aspect	Thread	Process
Memory	Shared heap with other threads	Own separate memory space
Creation cost	Lightweight – fast to create	Heavyweight – slow to create
Communication	Direct shared memory	IPC (sockets, pipes, etc.)
Crash impact	Can crash whole process	Isolated – only process dies
Context switch	Cheaper	More expensive

★ **FREQUENTLY ASKED** Q2. What are the different ways to create a thread in Java?

Answer:

There are four main ways to create a thread in Java:

1. Extending Thread class

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Running in: " + Thread.currentThread().getName());
    }
}
// Usage: new MyThread().start();
```

2. Implementing Runnable interface (Preferred)

```
class MyRunnable implements Runnable {
    @Override public void run() { /* task logic */ }
}
// Usage: new Thread(new MyRunnable()).start();
```

3. Lambda (Java 8+)

```
Thread t = new Thread(() -> System.out.println("Lambda thread"));
t.start();
```

4. Callable + Future (returns a value)

```
Callable<Integer> task = () -> 42;
ExecutorService es = Executors.newSingleThreadExecutor();
Future<Integer> future = es.submit(task);
int result = future.get(); // blocks until done
```

✓ Best Practices

- ✓ Prefer Runnable / Callable over extending Thread – keeps class hierarchy free for other bases
- ✓ Always use start(), never run() – run() executes on the calling thread, not a new one
- ✓ Use ExecutorService for production code instead of raw Thread creation

Q3. Explain the lifecycle / states of a Thread in Java.

Answer:

A thread passes through six states defined in **Thread.State** enum:

- **NEW** – Thread object created, **start()** not yet called
- **RUNNABLE** – Executing or ready to execute (waiting for CPU time slice)
- **BLOCKED** – Waiting to acquire a monitor lock (synchronized block)
- **WAITING** – Indefinitely waiting (wait(), join(), LockSupport.park())
- **TIMED_WAITING** – Waiting for a specified time (sleep(n), wait(n), join(n))
- **TERMINATED** – run() has completed (normally or via exception)

```
Thread t = new Thread(() -> {}); // NEW
t.start(); // → RUNNABLE
// inside run: Thread.sleep(1000) → TIMED_WAITING
// on synchronized block wait → BLOCKED
// run() ends → TERMINATED
```

★ FREQUENTLY ASKED Q4. What is the difference between start() and run() in Java threads?

Answer:

- **start()** – Creates a NEW thread by the JVM and executes run() on that thread. Correct approach.
- **run()** – Simply calls the run method on the CURRENT thread. No new thread is spawned. A common bug.

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()));
t.run();    // prints 'main' - no new thread!
t.start();  // prints 'Thread-0' - correct!
```

★ FREQUENTLY ASKED Q5. What is the difference between sleep() and wait()?

Answer:

Aspect	sleep()	wait()
Class	Thread	Object
Lock release	Does NOT release the lock	Releases the monitor lock
Wakeup	After specified duration	By notify() / notifyAll()
Context	Any context	Must be inside synchronized block
Interruption	Throws InterruptedException	Throws InterruptedException

Q6. What is daemon thread? When would you use it?

Answer:

A **daemon thread** is a background thread that does not prevent the JVM from exiting. When only daemon threads remain, the JVM shuts down. Created by calling **setDaemon(true)** before **start()**.

```
Thread t = new Thread(() -> { while(true) { /* background task */ } });
t.setDaemon(true);    // must be called BEFORE start()
t.start();
```

Use cases: garbage collection, log flushing, heartbeat monitors, resource cleanup.

Q7. What is Thread priority? Does it guarantee execution order?

Answer:

Thread priority is a hint to the OS scheduler (1 = MIN_PRIORITY, 5 = NORM_PRIORITY, 10 = MAX_PRIORITY). It does NOT guarantee execution order – the OS thread scheduler has ultimate control and behavior is platform-specific.

```
t.setPriority(Thread.MAX_PRIORITY);    // 10
```

✗ Common Mistakes

- ✗ Relying on thread priority for correctness – it is merely a hint
- ✗ Setting high priority and expecting it to always run first

Section 2: Synchronization

★ FREQUENTLY ASKED Q8. What is synchronization and why is it needed?

Answer:

Synchronization is the mechanism that controls access to shared resources by multiple threads, preventing race conditions and ensuring data consistency. It is needed because thread scheduling is non-deterministic and multiple threads can interleave in unpredictable ways.

Example – race condition without sync:

```
int counter = 0;
// Thread 1 and Thread 2 both run: counter++;
// Read-Increment-Write is NOT atomic → lost updates!
```

Fix with synchronized:

```
synchronized(this) { counter++; }
```

Q9. Explain the synchronized keyword – methods vs blocks.

Answer:

Synchronized instance method – lock on 'this'

```
public synchronized void increment() { count++; }
```

Synchronized static method – lock on Class object

```
public static synchronized void reset() { count = 0; }
```

Synchronized block – finer-grained, preferred

```
public void add(int val) {
    synchronized(lock) {    // lock is a dedicated Object
        count += val;
    }
    // code here runs without holding the lock
}
```

✓ Best Practices

- ✓ Prefer synchronized blocks over methods – minimize the critical section
- ✓ Never lock on 'this' in public APIs – callers can deadlock you externally
- ✓ Use a private final Object as the lock object

★ **FREQUENTLY ASKED** Q10. What is a deadlock? How do you detect and prevent it?

Answer:

Deadlock occurs when two or more threads each hold a lock the other needs and both are permanently blocked waiting.

```
// Thread 1: synchronized(A) { synchronized(B) { ... } }
// Thread 2: synchronized(B) { synchronized(A) { ... } } → DEADLOCK
```

Four Coffman conditions (ALL must hold for deadlock):

- Mutual Exclusion – resource held exclusively
- Hold and Wait – thread holds one resource while waiting for another
- No Preemption – locks cannot be forcibly taken away
- Circular Wait – circular chain of threads waiting for each other

Prevention strategies:

- **Lock ordering** – always acquire locks in the same global order
- **tryLock with timeout** – use `ReentrantLock.tryLock(timeout)` to avoid indefinite waiting
- **Lock-free data structures** – use `java.util.concurrent` classes
- **Single lock strategy** – avoid acquiring multiple locks where possible

Detection: `jstack <pid>` prints a thread dump identifying deadlocked threads.

Q11. What is livelock and starvation?

Answer:

- **Livelock:** Threads continuously change state in response to each other but never make progress (both keep retrying simultaneously).
- **Starvation:** A thread never gets CPU time because higher-priority threads or greedy threads monopolize resources. Fix with fair locks (`new ReentrantLock(true)`).

★ **FREQUENTLY ASKED** Q12. Explain the volatile keyword in Java.

Answer:

volatile guarantees:

- **Visibility** – writes to a volatile variable are immediately visible to all threads (no CPU caching)
- **Ordering** – prevents instruction reordering around the volatile access
- **NOT atomicity** – `volatile int x; x++;` is still a race condition!

```
private volatile boolean running = true;
// Thread 1 reads 'running' - always sees latest value
```

```
// Thread 2 sets running = false - immediately visible
```

Use volatile for: simple flags, double-checked locking (Java 5+), published references.

★ FREQUENTLY ASKED Q13. What is the Java Memory Model (JMM)?

Answer:

The JMM defines the rules for how threads interact through memory. Key concepts:

- **happens-before** – If action A happens-before B, then A's writes are visible to B
- **Main Memory vs Working Memory** – Each thread has its own working memory (cache); volatile/synchronized force sync with main memory
- **Reordering** – CPU and compiler may reorder instructions for performance; JMM constrains this

happens-before rules:

- Program order – each action in a thread happens-before the next in that thread
- Monitor lock – unlock happens-before subsequent lock on the same monitor
- volatile – write to volatile happens-before subsequent read of same variable
- Thread start – Thread.start() happens-before any action in the started thread
- Thread join – all actions in a thread happen-before Thread.join() returns

Section 3: java.util.concurrent Utilities

★ FREQUENTLY ASKED Q14. What is ReentrantLock? How does it differ from synchronized?

Answer:

Feature	synchronized	ReentrantLock
Fairness	Not configurable	new ReentrantLock(true)
tryLock	Not available	tryLock(timeout)
Condition vars	wait()/notify()	lock.newCondition()
Interruptible	No	lockInterruptibly()
Explicit unlock	Not needed	MUST call unlock() in finally

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try { /* critical section */ }
finally { lock.unlock(); } // ALWAYS unlock in finally!
```

Q15. What is ReadWriteLock and when should you use it?

Answer:

ReadWriteLock (ReentrantReadWriteLock) allows multiple concurrent readers but exclusive writers. Ideal for read-heavy workloads like caches.

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();
// Multiple threads can read concurrently:
rwLock.readLock().lock(); try { /* read */ } finally { rwLock.readLock().unlock(); }
// Only one thread can write:
rwLock.writeLock().lock(); try { /* write */ } finally
{ rwLock.writeLock().unlock(); }
```

★ **FREQUENTLY ASKED** Q16. Explain Atomic classes. What operations are they useful for?

Answer:

java.util.concurrent.atomic classes use CPU-level CAS (Compare-And-Swap) for lock-free thread safety.

- **AtomicInteger, AtomicLong, AtomicBoolean** – for primitive types
- **AtomicReference<V>** – for object references
- **AtomicIntegerArray, AtomicLongArray** – for arrays
- **LongAdder / LongAccumulator** – better performance under high contention

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // atomic counter++
counter.compareAndSet(expected, update); // CAS
```

Q17. What are the concurrent collections? When to prefer them over synchronized wrappers?

Answer:

Class	Use Case	Notes
ConcurrentHashMap	Thread-safe Map	Segment/bucket level locking
CopyOnWriteArrayList	Read-heavy List	Copies on every write – expensive
CopyOnWriteArraySet	Read-heavy Set	Based on CopyOnWriteArrayList
LinkedBlockingQueue	Producer-consumer	Bounded/unbounded blocking
PriorityBlockingQueue	Priority task queue	Unbounded, thread-safe
ConcurrentLinkedQueue	Non-blocking queue	Lock-free CAS operations
BlockingDeque	Double-ended blocking queue	Both ends can block

Use concurrent collections over **Collections.synchronizedXxx()** – they offer better throughput through fine-grained locking.

★ **FREQUENTLY ASKED** Q18. Explain CountdownLatch, CyclicBarrier, and Semaphore.

Answer:

CountDownLatch – wait for N events to complete

```
CountDownLatch latch = new CountDownLatch(3);  
// 3 worker threads each call latch.countDown()  
latch.await(); // main thread waits until count reaches 0
```

Note: One-time use – cannot be reset.

CyclicBarrier – all threads wait at a common point

```
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("All  
arrived!"));  
// Each thread calls barrier.await() - all block until 3 have arrived
```

Note: Reusable – resets automatically after each cycle.

Semaphore – control concurrent access count

```
Semaphore sem = new Semaphore(5); // max 5 concurrent accesses  
sem.acquire(); // blocks if 0 permits  
try { /* limited resource */ } finally { sem.release(); }
```

Q19. What is Phaser? How does it improve on CountDownLatch and CyclicBarrier?

Answer:

Phaser is a flexible barrier that supports dynamic registration of parties, multiple phases, and hierarchical composition. Unlike CyclicBarrier (fixed parties) and CountDownLatch (one-shot), Phaser allows parties to register/deregister dynamically.

```
Phaser phaser = new Phaser(3); // 3 parties  
phaser.arriveAndAwaitAdvance(); // similar to barrier.await()  
phaser.arriveAndDeregister(); // leave the phaser
```

Section 4: Executor Framework & Thread Pools

★ FREQUENTLY ASKED Q20. What is the Executor framework? Why use it instead of raw threads?

Answer:

The Executor framework (java.util.concurrent) separates task submission from thread management, enabling thread reuse, resource control, and richer lifecycle management.

- **Avoids thread creation overhead** – threads are reused from a pool
- **Resource control** – cap the number of concurrent threads

- **Task queue** – tasks queue up when all threads are busy
- **Lifecycle management** – orderly shutdown, await termination

★ **FREQUENTLY ASKED** Q21. Explain the different types of thread pools in Executors factory.

Answer:

Factory Method	Behavior	When to Use
<code>newFixedThreadPool(n)</code>	n threads, unbounded queue	Stable, predictable load
<code>newCachedThreadPool()</code>	Grows/shrinks, 60s idle TTL	Many short-lived tasks
<code>newSingleThreadExecutor()</code>	1 thread, sequential	Ordered, serial tasks
<code>newScheduledThreadPool(n)</code>	Scheduled / periodic tasks	Cron-like scheduling
<code>newWorkStealingPool()</code>	ForkJoinPool, parallelism level	Parallel divide-and-conquer

✗ **Common Mistakes**

- ✗ Using `newCachedThreadPool()` for long-running tasks – can create unlimited threads and cause OOM
- ✗ Using `newFixedThreadPool` with an unbounded queue – can cause unbounded task accumulation

★ **FREQUENTLY ASKED** Q22. How does `ThreadPoolExecutor` work internally?

Answer:

ThreadPoolExecutor is the backbone of all Executors. Key constructor parameters:

```
new ThreadPoolExecutor(
    corePoolSize,        // min threads kept alive
    maximumPoolSize,     // max threads allowed
    keepAliveTime,       // idle time before non-core thread dies
    unit,                // time unit
    workQueue,           // task queue
    threadFactory,       // how threads are created
    rejectionHandler     // what to do when full
)
```

Execution flow:

- If threads < corePoolSize → create new thread
- If threads >= core → put in workQueue
- If queue full and threads < max → create new thread
- If queue full and threads = max → execute RejectionHandler

Rejection policies:

- **AbortPolicy** (default) – throws `RejectedExecutionException`

- **CallerRunsPolicy** – caller thread executes the task (slows producer naturally)
- **DiscardPolicy** – silently drops the task
- **DiscardOldestPolicy** – drops oldest queued task, retries submission

Q23. What is the difference between submit() and execute() in ExecutorService?

Answer:

- **execute(Runnable)** – fire-and-forget; no return value; throws unchecked exception directly
- **submit(Runnable/Callable)** – returns a Future; exceptions captured in future.get(); safer for error handling

```
Future<?> f1 = executor.submit(runnable);    // Future<Void>
Future<Integer> f2 = executor.submit(callable);
executor.execute(runnable);                  // no Future returned
```

Section 5: Future, CompletableFuture & Async Programming

★ FREQUENTLY ASKED Q24. What are the limitations of Future?

Answer:

- **Blocking get()** – future.get() blocks the calling thread, defeating async purpose
- **No chaining** – cannot chain multiple async operations easily
- **No combine** – cannot combine results of multiple Futures without blocking
- **No exception handling** – no callback on failure; exceptions wrapped in ExecutionException
- **Cannot complete manually** – cannot set result from outside the computation

★ FREQUENTLY ASKED Q25. Explain CompletableFuture and its key methods.

Answer:

CompletableFuture<T> (Java 8+) implements both **Future<T>** and **CompletionStage<T>**, providing a rich non-blocking API for async composition.

Creating

```
CompletableFuture.runAsync(() -> { /* no result */ });
CompletableFuture.supplyAsync(() -> fetchData(), executor);
```

Transforming

```
cf.thenApply(result -> result.toUpperCase())    // sync transform
    .thenApplyAsync(s -> callApi(s), executor)    // async transform
```

```
.thenAccept(s -> System.out.println(s))    // consume, no return
.thenRun(() -> System.out.println("done")); // no input/output
```

Combining

```
cf1.thenCombine(cf2, (r1, r2) -> r1 + r2) // combine two CFs
CompletableFuture.allOf(cf1, cf2, cf3)    // wait for ALL
CompletableFuture.anyOf(cf1, cf2, cf3)    // first to complete
```

Error Handling

```
cf.exceptionally(ex -> defaultValue)
cf.handle((result, ex) -> ex != null ? fallback : result)
```

Q26. What is the ForkJoinPool and how does work stealing work?

Answer:

ForkJoinPool (Java 7+) is optimized for recursive divide-and-conquer tasks. Each thread has its own deque; idle threads **steal tasks** from the tail of busy threads' deques.

```
class SumTask extends RecursiveTask<Long> {
    protected Long compute() {
        if (size <= THRESHOLD) return computeDirectly();
        SumTask left = new SumTask(data, start, mid);
        left.fork(); // async execution
        SumTask right = new SumTask(data, mid, end);
        return right.compute() + left.join(); // join result
    }
}
```

Section 6: Classic Multithreading Problems

★ FREQUENTLY ASKED Q27. Implement the Producer-Consumer problem.

Answer:

Using BlockingQueue (Recommended)

```
BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(10);
```

```
// Producer
Runnable producer = () -> {
    for (int i = 0; i < 100; i++) {
        queue.put(i); // blocks if queue is full
    }
};
```

```
// Consumer
Runnable consumer = () -> {
    while (true) {
        Integer item = queue.take(); // blocks if queue is empty
        process(item);
    }
};
```

★ FREQUENTLY ASKED Q28. Implement a Singleton class that is thread-safe.

Answer:

Double-Checked Locking (Java 5+, requires volatile)

```
public class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) { // first check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) { // second check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Initialization-on-demand holder (Best – lazy + thread-safe + no sync)

```
public class Singleton {
    private static class Holder { static final Singleton INSTANCE = new Singleton(); }
    public static Singleton getInstance() { return Holder.INSTANCE; }
}
```

Q29. What is the Dining Philosophers problem and how to solve it?

Answer:

Five philosophers sit at a table. Each needs two forks to eat. Deadlock can occur if each picks up the left fork first.

Solutions:

- **Ordered resource acquisition** – philosopher N picks min-fork first, max-fork second
- **Waiter / arbiter** – a semaphore limits concurrent eaters to N-1
- **Chandy/Misra algorithm** – forks passed as messages, no shared state

Q30. Implement a thread-safe blocking queue from scratch.

Answer:

```
class BoundedQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;
    BoundedQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait(); // full - wait
        queue.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait(); // empty - wait
        T item = queue.poll();
        notifyAll();
        return item;
    }
}
```

✓ Best Practices

- ✓ Always use while (not if) for wait() checks – guards against spurious wakeups
- ✓ Use notifyAll() instead of notify() unless you know exactly one waiter needs to wake

Section 7: Advanced Topics

★ FREQUENTLY ASKED Q31. What is ThreadLocal? How and when should you use it?

Answer:

ThreadLocal<T> provides per-thread variable instances. Each thread gets its own copy, completely isolated from other threads.

```
ThreadLocal<SimpleDateFormat> formatter =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

// Each thread gets its own SimpleDateFormat instance - thread safe!
String date = formatter.get().format(new Date());
```

Use cases: User session context in web servers, database connections, SimpleDateFormat (not thread-safe), request-scoped data.

✗ Common Mistakes

- ✗ Not calling `ThreadLocal.remove()` in thread pool threads – causes memory leaks because threads are reused
- ✗ Storing heavy objects in `ThreadLocal` unnecessarily

★ FREQUENTLY ASKED Q32. Explain `StampedLock` and optimistic reading.

Answer:

`StampedLock` (Java 8+) improves read scalability with three modes:

- Writing** – exclusive write lock (like `writeLock`)
- Reading** – shared read lock (like `readLock`)
- Optimistic Reading** – lock-free read; validate after; retry if invalidated

```
StampedLock sl = new StampedLock();
long stamp = sl.tryOptimisticRead(); // no lock taken
int value = sharedValue;             // read
if (!sl.validate(stamp)) {           // check if write occurred
    stamp = sl.readLock();            // fall back to read lock
    try { value = sharedValue; }
    finally { sl.unlockRead(stamp); }
}
```

★ FREQUENTLY ASKED Q33. What are virtual threads (Project Loom)? How do they differ from platform threads?

Answer:

Virtual threads (Java 21 GA) are lightweight threads managed by the JVM, not the OS. They are designed for high-throughput I/O-bound workloads.

Aspect	Platform Thread	Virtual Thread
Stack size	~1 MB OS stack	Small heap object, grows dynamically
Creation cost	Thousands per JVM	Millions per JVM
Scheduling	OS scheduler	JVM ForkJoinPool scheduler
Blocking I/O	Blocks OS thread	Unmounts, carrier thread freed
API	<code>Thread.start()</code>	<code>Thread.ofVirtual().start(runnable)</code>

```
// Create virtual thread (Java 21)
Thread vt = Thread.ofVirtual().name("vthread-1").start(() -> doWork());
ExecutorService es = Executors.newVirtualThreadPerTaskExecutor();
```

Q34. What is the happens-before relationship in detail?

Answer:

happens-before is the formal JMM guarantee that if action A happens-before action B, then all writes made by A (and anything that happened before A) are visible to B.

Establishing happens-before:

- Thread.start() → all actions in the started thread
- Thread.join() → all actions in the joined thread happen-before join() returns
- Unlock of monitor → subsequent lock of same monitor
- Write to volatile → subsequent read of same volatile
- Actions on objects passed to executor → their execution

★ FREQUENTLY ASKED Q35. How does double-checked locking work and why is volatile required?

Answer:

Without volatile, the JVM may reorder: (1) allocate memory, (2) assign reference, (3) initialize object. Another thread can see a non-null but incompletely initialized object. volatile prevents this reordering by establishing a happens-before between write and read.

Without volatile (BROKEN):

```
private static Singleton instance; // No volatile - BROKEN
// Thread 2 might see partially initialized object!
```

With volatile (CORRECT):

```
private static volatile Singleton instance; // volatile REQUIRED
```

Q36. What is CAS (Compare-And-Swap)? How does it relate to atomic classes?

Answer:

CAS is a CPU-level atomic instruction: **compareAndSwap(variable, expected, new)** – sets variable to new only if it equals expected. Returns true if successful.

- **Non-blocking** – threads retry instead of blocking
- **ABA problem** – value changed from A to B back to A; CAS succeeds incorrectly. Use AtomicStampedReference.

```
AtomicInteger ai = new AtomicInteger(5);
boolean updated = ai.compareAndSet(5, 10); // true
```

Q37. What is the difference between notify() and notifyAll()?

Answer:

- **notify()** – wakes ONE arbitrary thread waiting on the monitor. Risky if multiple thread types wait.
- **notifyAll()** – wakes ALL waiting threads; they compete for the lock. Safer, slightly less efficient.

Rule: Use notifyAll() unless you are certain only one type of thread waits and exactly one needs to wake.

★ **FREQUENTLY ASKED** Q38. What is interrupt() and how do you handle interruption properly?

Answer:

Thread.interrupt() sets the interrupt flag of the target thread. Blocking methods (sleep, wait, join) throw **InterruptedException** when interrupted.

```
// Correct pattern:
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // RESTORE the flag!
    return; // or rethrow
}
```

✗ **Common Mistakes**

- ✗ Swallowing InterruptedException without restoring the flag – breaks cooperative cancellation
- ✗ Checking Thread.interrupted() in a loop without resetting (it clears the flag)

Q39. Explain the Exchanger class.

Answer:

Exchanger<V> allows two threads to exchange objects at a synchronization point. Both threads block until the other arrives, then swap.

```
Exchanger<List<Integer>> exchanger = new Exchanger<>();
// Thread 1 (producer):
List<Integer> full = fillBuffer();
List<Integer> empty = exchanger.exchange(full);
// Thread 2 (consumer):
List<Integer> full = exchanger.exchange(emptyBuffer);
```

Q40. What are memory leaks in multithreaded applications and how to prevent them?

Answer:

- **ThreadLocal leaks** – not calling remove() in thread pool threads. Fix: always remove() in finally.
- **Listener leaks** – registering listeners that hold references to large objects. Fix: use WeakReference listeners.
- **Unclosed ExecutorService** – threads kept alive indefinitely. Fix: always shutdown() and awaitTermination().
- **Task queue overflow** – unbounded task queues grow without limit. Fix: use bounded queues.

Section 8: Common Mistakes & Best Practices

8.1 Common Mistakes

Mistake	Correct Approach
Calling <code>run()</code> instead of <code>start()</code>	Always call <code>start()</code> to create a new thread
Using <code>if</code> for <code>wait()</code> checks	Use <code>while</code> loop – protects against spurious wakeups
Not unlocking in <code>finally</code>	Always unlock <code>ReentrantLock</code> in a <code>finally</code> block
Ignoring <code>InterruptedException</code>	Restore interrupt flag: <code>Thread.currentThread().interrupt()</code>
Not calling <code>ThreadLocal.remove()</code>	Call <code>remove()</code> in thread pool threads to prevent leaks
<code>volatile</code> for compound operations	Use <code>AtomicInteger</code> for <code>i++</code> or use <code>synchronized</code>
Synchronizing on <code>String</code> literals	Use private final <code>Object</code> lock – strings may be shared
Double-checked locking without <code>volatile</code>	Always use <code>volatile</code> for DCL singleton
Not shutting down <code>ExecutorService</code>	Always call <code>shutdown()</code> and <code>awaitTermination()</code>
Using <code>Thread.stop()</code> / <code>suspend()</code>	Use a <code>volatile</code> flag or <code>interrupt()</code> for cooperative cancellation

8.2 Best Practices Summary

✓ Best Practices

- ✓ Prefer higher-level concurrency utilities over low-level `wait/notify`
- ✓ Use immutable objects wherever possible – they are inherently thread-safe
- ✓ Minimize the scope of `synchronized` blocks – lock only what you must
- ✓ Prefer concurrent collections (`ConcurrentHashMap`) over `synchronized` wrappers
- ✓ Document thread safety of every class clearly (`@ThreadSafe`, `@NotThreadSafe`)
- ✓ Use `ExecutorService`; shut it down in a `try-finally` or shutdown hook
- ✓ Prefer stateless objects (all data in local variables) – always thread-safe
- ✓ Name your threads for debugging: `new Thread(task, 'OrderProcessor-1')`
- ✓ Use thread dumps (`jstack`) and profilers to diagnose concurrency issues
- ✓ Test concurrent code with stress tests and tools like `JCStress`

Section 9: Quick-Fire Interview Questions (Q41–Q60)

★ FREQUENTLY ASKED Q41. What is a race condition?

Answer:

When the outcome depends on the relative timing/interleaving of threads accessing shared state.

Q42. Is String thread-safe in Java?

Answer:

Yes – String is immutable, so it is inherently thread-safe.

Q43. Is HashMap thread-safe?

Answer:

No. Use ConcurrentHashMap or Collections.synchronizedMap() instead.

★ FREQUENTLY ASKED Q44. Is ArrayList thread-safe?

Answer:

No. Use CopyOnWriteArrayList or Collections.synchronizedList().

Q45. What is a monitor in Java?

Answer:

Every Java object has an associated monitor lock used by synchronized. Only one thread can hold the monitor at a time.

Q46. Difference: Callable vs Runnable?

Answer:

Callable can return a value and throw checked exceptions. Runnable cannot.

★ FREQUENTLY ASKED Q47. What is lock striping?

Answer:

Splitting a lock into multiple finer-grained locks (e.g., ConcurrentHashMap uses segment/bucket locking).

Q48. What is busy-waiting / spin-lock?

Answer:

A loop continuously checking a condition. Wastes CPU. Prefer blocking with wait() or LockSupport.

Q49. What is a memory barrier?

Answer:

A CPU/compiler instruction that prevents reordering across the barrier. volatile and synchronized insert barriers.

★ **FREQUENTLY ASKED** Q50. What is false sharing?

Answer:

Two threads write to variables that reside on the same CPU cache line, causing unnecessary cache invalidation. Fix with padding or `@Contended`.

Q51. What is park/unpark?

Answer:

`LockSupport.park()` suspends a thread; `unpark(thread)` resumes it. The basis for higher-level locks.

Q52. Can a static method be synchronized?

Answer:

Yes – it locks on the Class object of the class.

★ **FREQUENTLY ASKED** Q53. Can constructors be synchronized?

Answer:

No – a constructor cannot be synchronized; the object isn't yet visible to other threads.

Q54. What is optimistic locking?

Answer:

Assume no conflict; read without locking, then validate. If conflict, retry. CAS and `StampedLock` optimistic read use this.

Q55. What is a Condition variable?

Answer:

A condition associated with a `ReentrantLock`. `Lock.newCondition()` provides `await/signal` analogous to `wait/notify` but more flexible.

★ **FREQUENTLY ASKED** Q56. What is the volatile write guarantee?

Answer:

A write to a volatile variable flushes to main memory immediately and establishes a happens-before for subsequent readers.

Q57. What is thread confinement?

Answer:

Ensuring an object is accessed by only one thread. Eliminates synchronization need. `ThreadLocal`

and stack confinement are examples.

Q58. What is immutability and why is it helpful?

Answer:

An immutable object's state cannot change after construction → inherently thread-safe with no synchronization needed.

Q59. What does shutdown() vs shutdownNow() do?

Answer:

shutdown(): stop accepting new tasks, wait for running ones. shutdownNow(): interrupt running threads and return pending tasks.

Q60. How do you detect a deadlock at runtime?

Answer:

jstack <pid> prints a thread dump; JVM highlights threads in deadlock. Also programmable via ThreadMXBean.findDeadlockedThreads().

Cheat Sheet: Java Multithreading at a Glance

Thread Creation Methods

Method	When to Use
implements Runnable	General use; keeps class hierarchy free
extends Thread	Rarely; only if you need to override Thread behaviour
Callable + Future	When you need a result or exception propagation
Lambda / method ref	Short tasks in Java 8+
ExecutorService.submit()	Production code; pooled, managed execution
CompletableFuture.supplyAsync()	Async pipelines with chaining
Thread.ofVirtual() (Java 21)	I/O-bound tasks; millions of threads

Synchronization Toolkit

Tool	Key Trait	Use When
synchronized	Built-in, reentrant	Simple mutual exclusion
ReentrantLock	Trylock, fair, condition	Advanced lock control needed
ReadWriteLock	Multiple readers OR one writer	Read-heavy caches
StampedLock	Optimistic reads	High-read, low-write scenarios
volatile	Visibility only	Flags, simple published refs
Atomic classes	Lock-free CAS	Counters, accumulators

CountDownLatch	One-shot N-count	Wait for N tasks to start/finish
CyclicBarrier	Reusable, all-or-nothing	Iterative parallel phases
Semaphore	Permit-based rate limit	Connection pool, throttling
Phaser	Dynamic parties, multi-phase	Complex parallel algorithms

Thread Pool Quick Reference

Pool Type	Threads	Best For
FixedThreadPool(n)	Fixed n	CPU-bound, stable load
CachedThreadPool	0 to ∞	Many short I/O tasks
SingleThreadExecutor	1	Sequential ordering
ScheduledThreadPool	Fixed n	Cron, periodic tasks
WorkStealingPool	Parallelism	Recursive, divide-and-conquer
VirtualThreadPerTask	Unlimited VT	Massive I/O concurrency (J21)

Key Terms & Formulas

Term	Definition
Race Condition	Output depends on non-deterministic thread interleaving
Critical Section	Code segment accessing shared resource – must be protected
Monitor	Per-object lock; one thread enters synchronized block at a time
happens-before	JMM guarantee: writes by A visible to B if A hb B
CAS	Atomic: set new value only if current == expected
Amdahl's Law	$\text{Speedup} \leq 1/(S + (1-S)/N)$ where S=serial fraction, N=threads
Thread Safety	Behaves correctly when accessed from multiple threads simultaneously
Reentrancy	Thread can re-acquire a lock it already holds
False Sharing	Cache-line contention between independently-modified variables
Memory Barrier	Instruction preventing CPU/compiler reordering

Pattern Cheat Sheet

Pattern	Implementation
Thread-safe Singleton	Initialization-on-demand holder idiom
Producer-Consumer	LinkedBlockingQueue.put() / take()
Parallel task + result	ExecutorService.invokeAll(callables)
Async pipeline	CompletableFuture.supplyAsync().thenApply().thenAccept()
Cancel long-running task	Set volatile boolean flag; check in loop; call interrupt()
Read-heavy cache	ConcurrentHashMap or ReadWriteLock
Throttle concurrency	Semaphore with N permits
Wait for multiple tasks	CountDownLatch or CompletableFuture.allOf()
Divide & conquer	ForkJoinPool + RecursiveTask

Interview Tips

✓ Best Practices

- ✓ Start simple – explain the basic problem before discussing optimized solutions
- ✓ Always mention thread safety when designing any shared data structure
- ✓ When asked about synchronization, discuss trade-offs: synchronized vs Lock vs Atomic
- ✓ Mention volatile + happens-before when discussing visibility problems
- ✓ Always name concurrent collections (ConcurrentHashMap) as an alternative to synchronized wrappers
- ✓ For any classic problem (producer-consumer, singleton), know both the low-level and high-level solutions
- ✓ Discuss deadlock prevention proactively – shows maturity
- ✓ Mention Java 21 virtual threads if talking about scalable I/O servers
- ✓ Know how to read a thread dump (jstack) – frequently asked in senior interviews

Good luck! ☐ ☐